Dormitory Management System

Design Compliance Report

Reference: Final Design.pdf (submitted 2026-01-27)

Module: Advanced Web Development — University of Hildesheim

Evaluated: 2026-03-15

This report compares the implemented Dormitory Management System against the Final Design submission. Each design artifact — Angular architecture, data model, behavioral design, and backend design — is assessed for compliance. Items are classified as Implemented, Partially Implemented, or Not Implemented, with justification drawn from the actual codebase.

1. ANGULAR ARCHITECTURE — COMPONENTS, SERVICES & MODULES

1.1 Components

The Final Design specifies eleven Angular components. The table below shows which were implemented as standalone files and which were merged into the dashboard components.

Designed Component	Status	Implementation Notes
AppComponent	Implemented	src/app/app.ts — root component with RouterOutlet shell. Matches design intent.
LoginComponent	Implemented	src/app/pages/login/ — authentication form calling AuthService. Matches design.
StudentDashboardComponent	Implemented	src/app/pages/student-dashboard/ — student self-service dashboard. Present and functional.
AdminDashboardComponent	Implemented	src/app/pages/dashboard/ — admin dashboard. Present and functional.
NavMenuComponent	Not Implemented	Design specifies a standalone NavMenuComponent for menu navigation. Navigation is embedded inside each dashboard component as a sidebar. No separate NavMenuComponent file exists.
ApplicationComponent	Not Implemented	Application form logic lives inside StudentDashboardComponent. No separate component file.
ContractComponent	Not Implemented	Contract display and upload logic is embedded in StudentDashboardComponent.
ComplaintComponent	Not Implemented	Complaint form and list are embedded in StudentDashboardComponent.
DocumentsComponent	Not Implemented	File list and download logic is embedded in StudentDashboardComponent.
ReceiptComponent	Not Implemented	Receipt upload logic is embedded in StudentDashboardComponent.
ProgressComponent	Not Implemented	Progress bar and polling logic are embedded in StudentDashboardComponent via reportModal.
RegisterComponent (not in design)	Added — Correct	src/app/pages/register/ — full student registration UI. Not in design; correctly added.
DashboardSectionComponent (not in design)	Added — Correct	Empty stub for Angular child routes /dashboard/* and /student-dashboard/*. Added to support router navigation.

1.2 Business Services

The Final Design specifies eight domain-specific services. All API calls are instead consolidated into one monolithic ApiService.

Designed Service	Status	Implementation Notes
AuthenticationService	Partially Implemented	AuthService in src/app/services/auth.service.ts handles login, logout, token storage, and role helpers. Missing: HTTP interceptor pattern — headers are set manually in ApiService.
ApplicationService	Not Implemented	No ApplicationService. All application API calls (getApplications, submitApplication, uploadApplicationFiles, updateApplicationStatus) are in the monolithic ApiService.
ContractService	Not Implemented	No ContractService. Contract API calls (getContracts, uploadSignedContract, downloadContract) are in ApiService.
ComplaintService	Not Implemented	No ComplaintService. Complaint API calls (getComplaints, submitComplaint, updateComplaintStatus) are in ApiService.
ReceiptService	Not Implemented	No ReceiptService. Payment and receipt API calls (recordPayment, getPayments, verifyPayment, downloadReceipt) are in ApiService.
FileService	Not Implemented	No FileService. File upload and download calls (uploadApplicationFiles, downloadFile, getMyFiles) are in ApiService.
ReportService	Not Implemented	No ReportService. Report generation, progress polling, and download calls are in ApiService.
ProgressService	Not Implemented	No ProgressService. Progress polling (setInterval) logic is embedded directly in StudentDashboardComponent.
ApiService (not in design)	Added	src/app/services/api.service.ts — monolithic service handling all 30+ API calls. Not in design, which specified separate domain services.
NotificationService (not in design)	Added	src/app/services/notification.service.ts — handles SSE connection, bell notifications, and toasts. Not in design; added as an extra feature.

1.3 Infrastructure Guards

Designed Guard	Status	Implementation Notes
AuthGuard	Implemented	src/app/guards/auth.guard.ts — functional CanActivateFn. Checks AuthService.isLoggedIn(). Redirects unauthenticated users to /login. Applied to both /dashboard and /student-dashboard.
RoleGuard	Implemented	src/app/guards/role.guard.ts — functional CanActivateFn. Reads route.data['role'], compares to AuthService.getRole(). Redirects wrong-role users to their correct dashboard.

1.4 Angular Modules

The design specifies four NgModules. The project uses Angular 21 standalone components instead — a valid modern pattern but a deviation from the design.

Designed Module	Status	Implementation Notes
AppModule	Not Implemented	Angular 21 standalone components bootstrapped via bootstrapApplication() in main.ts. No NgModule-based AppModule. Accepted modern Angular 17+ pattern.
CoreModule	Not Implemented	No CoreModule. Shared services (AuthService, ApiService, NotificationService) are @Injectable({ providedIn: "root" }) — achieves same singleton scope.
StudentModule	Not Implemented	No StudentModule. StudentDashboardComponent is a standalone component with imports: [CommonModule, FormsModule, RouterOutlet].
AdminModule	Not Implemented	No AdminModule. DashboardComponent is a standalone component with its own imports.

2. DATA MODEL — ENTITIES AND RELATIONSHIPS

2.1 Entity Compliance

The Final Design specifies eight core entities. All eight are implemented. Six additional tables were added beyond the design scope (driven by multi-dormitory support and extra features).

Designed Entity	Status	Implementation Notes
User	Implemented	users table: user_id, name, email, password_hash, role ENUM(STUDENT/ADMIN). Matches design. dormitory_id is an addition beyond single-dormitory scope.
StudentProfile	Implemented	student_profiles table: profile_id, user_id, room_id, phone, student_id_number, course, university, application_status. Matches designed attributes.
DormApplication	Implemented	dorm_applications table: application_id, user_id, submission_date, status ENUM(PENDING/ACCEPTED/REJECTED), assigned_room_id. Matches design. Additions: application_type, remarks.
Contract	Implemented	contracts table: contract_id, user_id, room_id, start_date, end_date, status ENUM(ACTIVE/EXTENDED/TERMINATED). Matches design. Additions: monthly_rent, due_day, generated_doc_file_id, signed_document_file_id.
RentPayment	Implemented	rent_payments table: payment_id, user_id, month, amount, receipt_path, created_at. Matches design fields. Addition: verification_status ENUM.
Complaint	Implemented	complaints table: complaint_id, user_id, description, status ENUM(SUBMITTED/IN_PROGRESS/RESOLVED), created_at. Matches design exactly.
Report	Implemented	reports table: report_id, user_id, file_path, created_at. Matches core design fields. Additions: status, progress_id, report_source ENUM.
FileMetadata	Implemented	file_metadata table: file_id, file_path, file_type, upload_date. Matches design. Additions: application_id, user_id for ownership tracking.
dormitories (not in design)	Added	Multi-dormitory support. Contains name, address, contact info, max_capacity, notification preference flags. Not in original single-dormitory design.
rooms (not in design)	Added	rooms table: room_id, room_number, floor, type, status ENUM(occupied/vacant/maintenance), resident_id. Rooms were implicit in design but not a separate entity.
termination_requests (not in design)	Added	Student-initiated contract termination request table with status ENUM(PENDING/ACCEPTED/REJECTED). Not in original design.
notifications (not in design)	Added	Real-time SSE notification storage: notification_id, user_id, type, message, tab, is_read, reference_id. Not in design.
progress (not in design)	Added	Backend long-running task progress tracking in DB. Design tracked progress in-memory; DB table is an addition.
admin_profiles (not in design)	Added	Admin profile table: user_id, dormitory_id, position, phone. Not in original design. Added for admin profile management.

2.2 Relationship Compliance

Designed Relationship	Status	Notes
User has exactly one role (STUDENT or ADMIN)	Implemented	role ENUM on users table. Enforced by requireRole() middleware.
User (STUDENT) has 0..1 StudentProfile	Implemented	student_profiles.user_id FK. One profile per student, inserted at registration.
User (STUDENT) can submit many DormApplications	Implemented	dorm_applications.user_id FK. Multiple applications per student supported.

Designed Relationship	Status	Notes
User (STUDENT) has at most one active Contract	Implemented	Enforced via transactional PATCH /api/applications/:id/status logic.
User (STUDENT) has many RentPayments	Implemented	rent_payments.user_id FK. Multiple payments per student.
User (STUDENT) has many Complaints	Implemented	complaints.user_id FK. Multiple complaints per student.
User can generate many Reports	Implemented	reports.user_id FK. Multiple reports per user.
DormApplication/Contract/RentPayment reference FileMetadata	Implemented	file_metadata.application_id for docs. contracts.generated_doc_file_id and signed_document_file_id FKs. rent_payments.receipt_path stores receipt reference.

3. BEHAVIORAL DESIGN — SEQUENCE DIAGRAM COMPLIANCE

The Final Design includes six sequence diagrams covering core workflows. Each is assessed below.

Sequence Diagram	Status	Implementation Notes
Login Scenario	Implemented	LoginComponent !' AuthService !' POST /api/login !' JWT !' authGuard + roleGuard !' redirect to correct dashboard. Matches design exactly.
Application Submission	Implemented (deviation)	Workflow is functionally correct. Deviation: ApplicationComponent logic is inside StudentDashboardComponent (not a separate component). FileService and ApplicationService logic is in ApiService. The sequence of actions matches; component/service decomposition does not.
Application Review & Decision	Implemented	Admin dashboard fetches applications via ApiService. acceptDialog and rejectDialog modals confirm before PATCH /api/applications/:id/status with transactional room assignment. Matches design workflow.
Complaint Submission & Processing	Implemented (deviation)	Complaint form is in StudentDashboardComponent (not a separate ComplaintComponent). complaintDialog confirms before API call. Admin manages status via PATCH /api/complaints/:id/status. Workflow matches; service/component decomposition does not.
Rent Payment Receipt Upload & Download	Implemented (deviation)	Receipt upload is in StudentDashboardComponent (not a separate ReceiptComponent). POST /api/payments with FormData. Admin downloads via GET /api/payments/:id/receipt. Workflow matches; service/component decomposition does not.
Report Generation with Progress	Implemented	reportModal triggers ApiService.generateReport() !' POST /api/reports !' setImmediate() starts async generation !' setInterval() polls GET /api/reports/:id/progress every 600ms !' progress bar updated !' download on COMPLETED. Technical pattern matches design exactly.

4. BACKEND DESIGN COMPLIANCE

Design Element	Status	Implementation Notes
Node.js + Express.js REST API	Implemented	server.js — single-file Express app on port 3000. 40+ REST endpoints with authMiddleware and requireRole(). Matches design.
MySQL relational database	Implemented	mysql2 driver. dorm_management database. All 8 designed tables plus 6 extra. Matches design.
JWT authentication + role-based access	Implemented	jsonwebtoken library. authMiddleware decodes token and attaches req.user. requireRole() enforces role-based access. Matches design.

Design Element	Status	Implementation Notes
Async long-running report generation	Implemented	POST /api/reports uses setImmediate() for non-blocking PDF generation. generatePdfReport() is non-blocking and updates progress table at milestones. Matches design.
Progress tracking via API	Implemented	GET /api/reports/:id/progress reads progress table and returns { percentage, status }. Frontend polls every 600ms. Matches design.
File upload and download	Partially Implemented	Multer disk storage to /uploads/ subdirectories. Design specified Google Firebase Storage — local disk used instead. Functionally equivalent for local dev; not production-ready as specified.
Google Firebase Storage	Not Implemented	Design specified Firebase for all file storage. Local disk storage via Multer is used throughout. This is the primary backend deviation from the Final Design.

5. DESIGN COMPLIANCE SUMMARY

5.1 Compliance by Area

Design Area	Designed	Implemented	Partially	Not Implemented
Angular Components	11	4	0	7 (merged into dashboards)
Business Services	8	0	1 (AuthService)	7 (merged into ApiService)
Infrastructure Guards	2	2	0	0
Angular NgModules	4	0	0	4 (standalone pattern used)
Data Entities	8	8	0	0 (plus 6 extra added)
Entity Relationships	8	8	0	0
Behavioral Workflows	6	5	1 (app submission)	0
Backend Technologies	6	5	0	1 (Firebase !' local disk)

5.2 Key Deviations from Design

Monolithic Components: Design specifies 7 separate feature components (ApplicationComponent, ContractComponent, ComplaintComponent, DocumentsComponent, ReceiptComponent, ProgressComponent, NavMenuComponent). All feature logic is embedded inside the two dashboard components. This is the largest structural deviation and results in very large component files.

Monolithic ApiService: Design specifies 7 domain services. A single ApiService handles all 30+ API calls. The domain separation intended by the design is absent. Each service group (application, contract, complaint, receipt, file, report, progress) is identifiable in ApiService but not separated.

No NgModules: Design specifies AppModule, CoreModule, StudentModule, AdminModule. Angular 21 standalone component pattern is used instead. This is a valid and recommended modern Angular approach but deviates from the NgModule-based design document.

Firebase !' Local Disk Storage: Design specified Google Firebase Storage for all user-uploaded files (contracts, receipts, application documents). Local disk storage via Multer (/uploads/ directory) is used. Functionally equivalent for a local development environment; not production-ready as specified.

Extra Tables Beyond Design Scope: Six tables added beyond the designed schema: dormitories, rooms, termination_requests, notifications, progress, admin_profiles. These are driven largely by multi-dormitory support and the real-time notification system — both features beyond requirements and design scope.

5.3 Correctly Implemented Design Elements

' All 8 core data entities with correct attributes and types in MySQL
' All 8 designed entity relationships correctly implemented
' AuthGuard and RoleGuard — implemented exactly as designed (functional CanActivateFn guards)
' JWT authentication with role-based access control on all endpoints
' Two-role system (STUDENT / ADMIN) with correct access separation
' Async long-running report generation (setImmediate) with DB progress tracking
' Frontend progress polling and visual progress bar in reportModal
' Dialog-based confirmations for complaint submission (R22) and application decisions (R23)
' Router-based menu navigation using Angular child routes

' File upload and download for all three file categories (contracts, application docs, receipts)
' All 6 behavioral workflows function end-to-end as described in the sequence diagrams